

Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 4 : catégories de produits

1. Ajouter les catégories de produits

Créez l'entité « Category » : `symfony console make:entity Category`

On ajoute ensuite le champ « name » (string, 255 caractères, Non Null).

```
created: src/Entity/Category.php
created: src/Repository/CategoryRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.

New property name (press <return> to stop adding fields):
> name

Field type (enter ? to see all types) [string]:
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Category.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

Next: When you're ready, create a migration with symfony.exe console make:migration
```

2. Modifier le champ « name » de l'entité « Category » pour qu'il soit unique.

Allez dans le fichier `src\Entity\Category.php` et modifiez les propriétés de la colonne « name » :

```
#[ORM\Column(length: 255, unique: true)]
private ?string $name = null;
```

3. Préparer la migration pour créer la table « Category » dans la BDD

`symfony console make:migration`

4. Exécuter la migration avec doctrine

`symfony console doctrine:migration:migrate`

5. Créer le contrôleur pour les catégories :

`symfony console make:controller CategoryController`

6. Implémenter l'ajout des catégories

Ouvrez le fichier `\src\Controller\CategoryController.php` et ajoutez le code suivant dans la classe « CategoryController » :

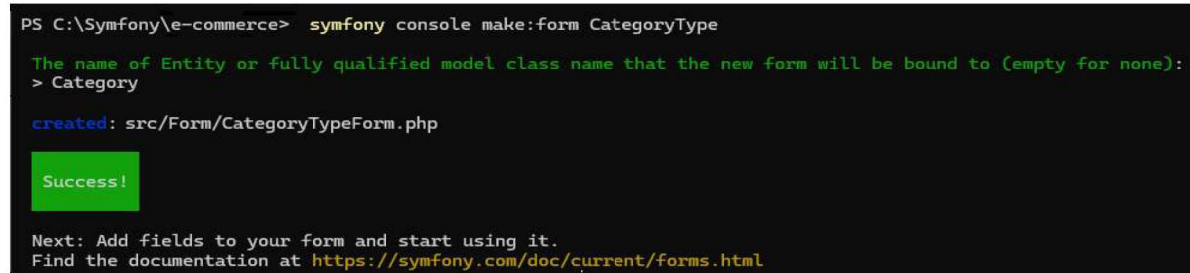
```
#[Route('/category/new', name: 'app_category_new')]
public function addCategory(EntityManagerInterface $entityManager): Response
{
    return $this->render('category/new.html.twig');
}
```

Ajoutez la vue en créant un nouveau fichier « Templates\category\new.html.twig ».

7. Créez un nouveau formulaire « CategoryType » en utilisant la commande :

`symfony console make:form`

Reliez le nouveau formulaire à l'entité « Category » :



```
PS C:\Symfony\e-commerce> symfony console make:form CategoryType

The name of Entity or fully qualified model class name that the new form will be bound to (empty for none):
> Category

created: src/Form/CategoryTypeForm.php

Success!

Next: Add fields to your form and start using it.
Find the documentation at https://symfony.com/doc/current/forms.html
```

8. Ajouter le formulaire au contrôleur.

Allez dans le fichier « src\controller\CategoryController.php » et ajoutez l'import de plusieurs classes :

```
use App\Entity\Category;
use App\Form\CategoryTypeForm;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\HttpFoundation\Request;
```

Modifiez le code de la fonction « addCategory » comme ci-dessous :

```
#[Route('/category/new', name: 'app_category_new')]
public function addCategory(EntityManagerInterface $entityManager, Request $request ):Response
{
    $category = new Category();

    $form = $this->createForm(CategoryTypeForm::class, $category);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()){
        $entityManager->persist($category);
        $entityManager->flush();
        return $this->redirectToRoute('app_category');
    }

    return $this->render('category/new.html.twig', ['categoryForm'=>
    $form->createView()]);
}
```

9. Modifier le code de la vue dans Templates\category\new.html.twig :

```
{% extends 'base.html.twig' %}

{% block title %}New Category{% endblock %}

{% block body %}
<div class = "container">
    <div class "form">
        <h2>Ajouter une catégorie</h2>
        <br>
        {{ form_start(categoryForm) }}
        {{ form_widget(categoryForm.name, {'attr':{'class':'form form-control','placeholder':"catégorie"}}) }}
        <br>
        <button type="submit" class="btn btn-primary">Enregistrer </button>
        {{ form_end(categoryForm) }}
    </div>
</div>
{% endblock %}
```

10. Modification des catégories.

Ouvrez le contrôleur src\controller\CategoryController.php et ajoutez la fonction updateCategory :

```
#[Route('/category/{id}/update', name: 'app_category_update')]
public function updateCategory(Category $category, EntityManagerInterface $entityManager,
Request $request):Response
{
    $form = $this->createForm(CategoryTypeForm::class, $category);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()){
        $entityManager->flush();
        return $this->redirectToRoute('app_category');
    }

    return $this->render('category/update.html.twig', ['categoryForm'=>
    $form->createView()]);
}
```

11. Créez la vue Templates\category\update.html.twig :

```
{% extends 'base.html.twig' %}
```

```

{% block title %}Update Category{% endblock %}

{% block body %}
<div class = "container">
    <div class "form">
        <h2>Modifier une catégorie</h2>
        <br>
        {{ form_start(categoryForm) }}
        {{ form_widget(categoryForm.name, {'attr':{'class':'form form-control','placeholder':"catégorie"}}) }}
        <br>
        <button type="submit" class="btn btn-primary">Modifier </button>
        {{ form_end(categoryForm) }}
    </div>
</div>
{% endblock %}

```

12. Afficher toutes les catégories.

Commencez par modifier la route « /category » du contrôleur « CategoryController.php » :

```

#[Route('/category', name: 'app_category')]
public function index(CategoryRepository $categorieRepository): Response
{
    $categories = $categorieRepository->findAll();
    return $this->render('category/index.html.twig', [
        'categories' => $categories
    ]);
}

```

Modifiez ensuite la vue templates\category\index.html.twig :

```

{% extends 'base.html.twig' %}

{% block title %}categories{% endblock %}

{% block body %}
<div class = "container">
    <br>
    <h1>Catégories</h1>
    {% include 'layouts/_flash_message.html.twig' %}
    <br>
    <table class="table">
        <tr>
            <th>id</th>
            <th>nom de la catégorie</th>

```

```

        <th>actions</th>
    </tr>
    {% for category in categories %}
    <tr>
        <td>{{ category.id }}</td>
        <td>{{ category.name }}</td>
        <td>
            <a href="{path('app_category_update',{'id':category.id})}">Modifier</a>
        </td>
    </tr>
    {% else %}
    <p>Aucune catégorie à afficher</p>
    {% endfor %}
</table>
</div>
{% endblock %}

```

13. Supprimer une catégorie.

Ouvrez le contrôleur `src\controller\CategoryController.php` et ajoutez la fonction `deleteCategory` :

```

#[Route('/category/{id}/delete', name: 'app_category_delete')]
public function deleteCategory(Category $category, EntityManagerInterface
$entityManager, Request $request):Response
{
    $entityManager->remove($category);
    $entityManager->flush();

    return $this->redirectToRoute('app_category');
}

```

Ouvrez ensuite la vue `templates\category\index.html.twig` et ajoutez le lien pour aller vers la route « `app_category_delete` » :

```

{% extends 'base.html.twig' %}

{% block title %}categories{% endblock %}

{% block body %}
<div class = "container">
    <br>
    <h1>Catégories</h1>
    <br>
    <table class="table">
        <tr>
            <th>id</th>

```

```

        <th>nom de la catégorie</th>
        <th>actions</th>
    </tr>
    {% for category in categories %}
    <tr>
        <td>{{ category.id }}</td>
        <td>{{ category.name }}</td>
        <td>
            <a href="{{path('app_category_update',{'id':category.id})}}">
                Modifier</a> |
            <a href="{{ path('app_category_delete',{'id':category.id}) }}">
                Supprimer</a>
        </td>
    </tr>
    {% else %}
    <p>Aucune catégorie à afficher</p>
    {% endfor %}
</table>
</div>
{% endblock %}

```

14. Ajouter un bouton « nouvelle catégorie » dans la vue templates\category\index.html.twig

Ajoutez un lien vers la route 'app_category_new' après le tableau des catégories :

```

...
</table>
<a class="btn btn-outline-success" href="{{ path('app_category_new') }}">Nouvelle
catégorie</a>
</div>
{% endblock %}

```

15. Afficher des messages « Flash » après une action sur les catégories.

Créez un nouveau dossier « templates\layouts » puis un fichier « _flash_message.html.twig » dans ce dossier. Ajoutez dans « templates\layouts\ flash_message.html.twig » le code suivant :

```

{% for type, messages in app.flashes(['success', 'danger', 'info']) %}
    {% for message in messages %}
        <div class="col-md-12 mx-auto mt-2">
            <div class = "alert alert-{{ type }} alert-dismissible fade show">
                {{ message }}
            </div>
        </div>
    {% endfor %}
{% endfor %}

```

Ajoutez la ligne suivante dans le bloc « Body » du fichier « templates/category/index.html.twig » :

```
{% include 'layouts/_flash_message.html.twig' %}
```

Appelez ensuite les messages flash dans « \src\Controller\CategoryController.php » : ajoutez dans « addCategory() », « updateCategory() » et « deleteCategory() » la ligne
`$this->addFlash('success', 'La catégorie a été ajoutée');` juste avant la redirection (en adaptant le message, et le type). Exemples :

```
public function addCategory(EntityManagerInterface $entityManager, Request $request):Response
{
    $category = new Category();

    $form = $this->createForm(CategoryTypeForm::class, $category);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()){
        $entityManager->persist($category);
        $entityManager->flush();

        $this->addFlash('success', 'La catégorie a été ajoutée');
        return $this->redirectToRoute('app_category');
    }

    return $this->render('category/new.html.twig', ['categoryForm'=>
    $form->createView()]);
}
```

```
public function deleteCategory(Category $category, EntityManagerInterface $entityManager, Request $request):Response
{
    $entityManager->remove($category);
    $entityManager->flush();

    $this->addFlash('danger', 'La catégorie a été supprimée');
    return $this->redirectToRoute('app_category');
}
```

16. Amélioration du design pour la modification et la suppression des catégories.

Ouvrez la vue templates/category/index.html.twig et modifiez les lignes contenant les liens vers la modification et la suppression :

```
<a class="btn btn-outline-primary" href="{% path('app_category_update',
{'id':category.id}) %}">Modifier</a>
```

```
<a onclick="return confirm('Voulez-vous vraiment supprimer cette catégorie ?')"  
class="btn btn-danger" href="{{ path('app_category_delete',{'id':category.id})  
}}">Supprimer</a>
```

Ici, on utilise des classes Bootstrap pour afficher les liens sous forme de boutons et on utilise la fonction Javascript « onclick » pour afficher une fenêtre de confirmation lors de la suppression d'une catégorie.